
오픈소스SW 개발 프로젝트

「Kubernetes 기반 Claude AI 문서 요약 MSA 보고서」

8조

노경민, 신명섭, 이서현, 이온, 정민준

<목 차>

I. 서론

II. 주제 선정의 적절성

III. 조사 및 분석의 깊이

IV. 비교 분석 및 비판적 시각

V. 프로토타입 구현 및 검증

VI. 팀워크 및 기여도

I. 서론

1.1 프로젝트 개요

본 프로젝트는 Kubernetes 기반 Microservices Architecture(MSA)를 활용하여 AI 문서 요약 서비스를 안정적으로 운영할 수 있는 인프라를 구축하는 것을 목표로 한다. Docker를 이용해 서비스를 컨테이너화하고, Kubernetes 클러스터에서 자동 확장과 무중단 배포 기능을 실험적으로 구현하였다. 문서 요약 기능은 Claude API를 활용하여 실제 텍스트 요약 서비스를 제공한다.

1.2 선정 OSS 소개

Docker는 애플리케이션을 일관된 실행 환경으로 패키징하기 위해 사용되며, Kubernetes는 컨테이너 오케스트레이션을 통해 확장성·가용성을 보장하는 핵심 플랫폼이다. Claude API는 프로젝트의 주요 기능인 문서 요약을 수행하는 AI 엔진으로 채택하였다. 이 세 기술 조합은 현대 AI·클라우드 서비스 구축에 널리 사용되는 표준 스택이다.

1.3 보고서 구성 및 목적

본 보고서는 Kubernetes 기반 AI 문서 요약 MSA 구축 과정을 정리하고, 관련 OSS 기술 분석 및 실험 결과를 제시한다. 2장에서는 주제 선정 배경과 필요성을 설명하고, 3장에서는 Kubernetes·Docker·Claude API의 핵심 기술 요소를 분석한다. 4장에서는 구조적 비교와 평가를 수행하며, 5장에서는 구축한 시스템의 구성과 실험 결과를 제시한다.

II. 주제 선정의 적절성

2.1 도입 배경 및 필요성

최근 기업 서비스 환경은 트래픽 변동성 증가, 실시간 데이터 처리, 무중단 배포 요구, AI 기능 연동 등으로 인해 시스템 복잡도가 빠르게 증가하고 있다. 기존의 Monolithic 아키텍처는 다음과 같은 구조적 한계를 가진다.

- 확장성 부족: 특정 기능의 트래픽 증가가 전체 서버 증설로 이어짐
- 장애 전파성: 하나의 기능 장애가 전체 서비스 중단으로 연결

- 배포 불안정성: 새로운 기능 배포 시 전체 서비스를 재시작해야 함
- 자원 비효율성: 모든 기능을 동일한 스펙의 서버에서 실행해야 함

이에 따라 현대 산업은 Microservices Architecture(MSA) 기반의 시스템으로 전환하고 있으며, 이를 안정적으로 운영하기 위한 핵심 기술이 Kubernetes이다.

2.2 Kubernetes 도입 이유

1) 자동 확장(Auto scaling)

- CPU · 메모리 · 트래픽 지표 기반으로 Pod 수를 자동 조절
- 사용량이 급증해도 수동 개입 없이 대응 가능

2) 자가 치유(Self-Healing)

- 장애가 발생한 Pod를 자동으로 재시작하거나 교체
- 서비스 가용성(Availability)을 인위적 관리 없이 유지

3) 무중단 배포(Rolling Update)

- 업데이트 중에도 기존 Pod가 계속 서비스
- 웹 기반 AI 서비스에서 필수적인 안정성 확보

4) 자원 효율성(Resource Efficiency)

- 제한된 환경(예: 8GB RAM 로컬 클러스터)에서도 여러 컨테이너를 효율적으로 배치하여 운영 가능

2.3 산업적 연관성

• 국내 기업 도입 사례

- 카카오: 트래픽 급증 대응을 위해 Kubernetes 기반 자동 확장 · 자가 치유 체계를 구축하여 서비스 안정성 향상
- 쿠팡: 수백 개 마이크로서비스 운영 복잡성을 해결하기 위해 배포 자동화 및 통합 관리 체계를 Kubernetes로 구현
- 삼성전자: 온프레미스-클라우드 통합 운영을 위해 Kubernetes 기반 하이브리드 인프라 도입, 자원 활용률 및 운영 일관성 확보
- 네이버: 무중단 배포와 환경 일관성 확보를 위해 Kubernetes 기반 클라우드 네이티브 구조 구축

• 글로벌 기업 및 산업 동향

- Netflix, Spotify, Airbnb 등 대형 서비스 기업이 Kubernetes를 표준 인프라로 운영
- CNCF(2024): 전 세계 기업의 96%가 Kubernetes를 사용하거나 도입 검토 중
- AI · 검색 · 스트리밍 등 대규모 트래픽 기반 산업에서 필수 기술로 자리 잡음

Ⅲ. 조사 및 분석의 깊이

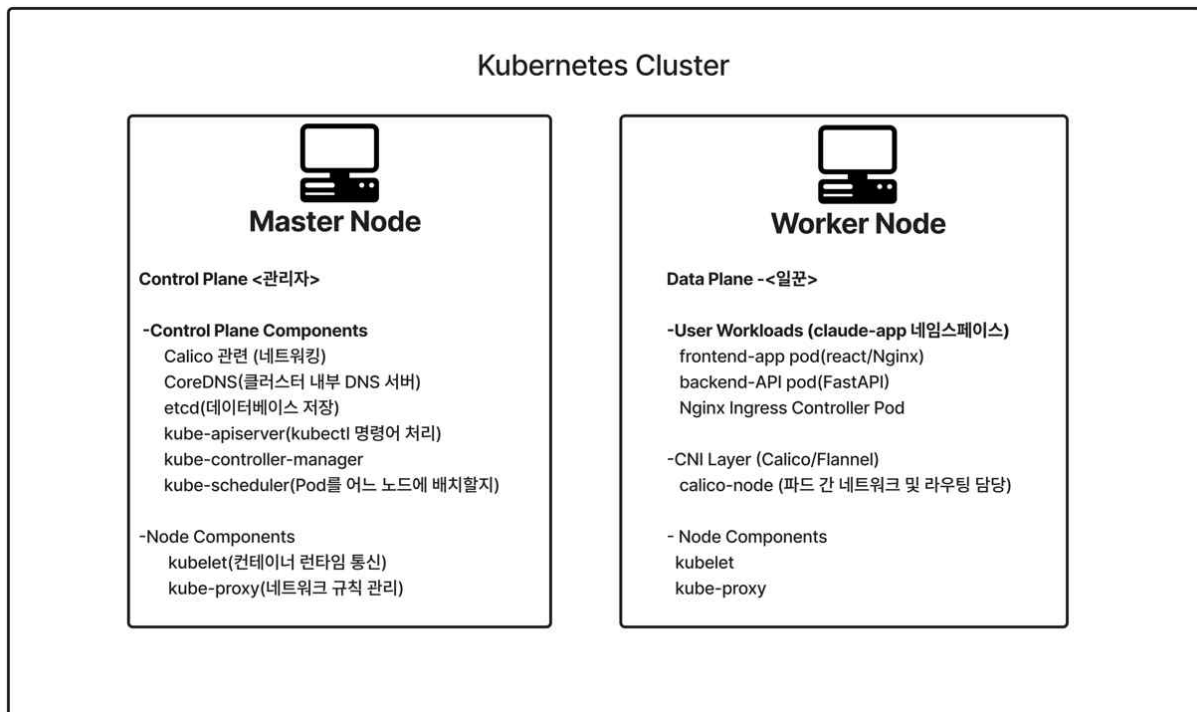
3.1 시스템 아키텍처

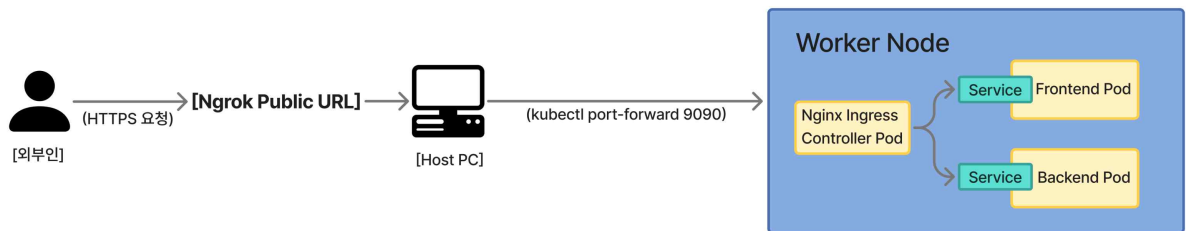
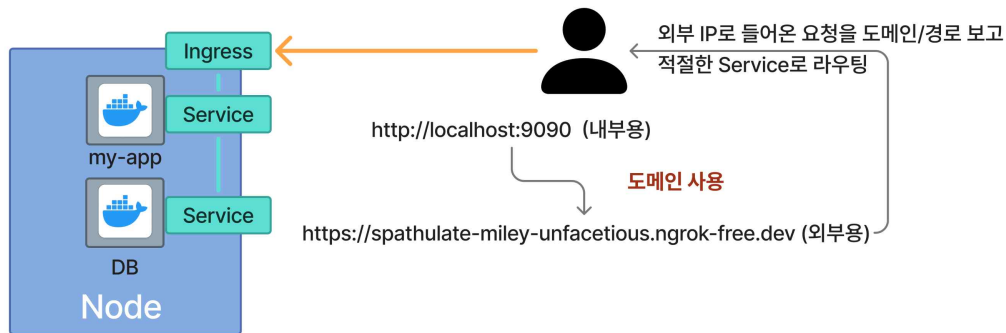
3.1.1 하드웨어 및 가상화 구성

구분	내용	기술적 의의
Host PC	Windows OS	개발 및 시연을 위한 온프레미스(On-Premise) 환경 제공
Virtualization	Oracle VirtualBox	경량의 가상 클라우드 환경 시뮬레이션
Network	NAT Network (10.100.0.0/24)	호스트 PC의 외부 네트워크 변화에 독립적인 고정적이고 일관된 내부 통신 환경 확보 (환경 일관성 확보)

3.1.2 Kubernetes 클러스터 토폴로지

- 구조: 1 Master Node, 1 Worker Node 구성
 - 기술적 의의: 고가용성(High Availability) 및 스케일 아웃(Scale-Out) 가능성 확보
- Master Node (10.100.0.104): Control Plane (API Server, Scheduler, Etcd) 운영 및 클러스터 관리
- Worker Node 1: 실제 서비스 컨테이너(Pod) 구동 및 Ingress Controller 트래픽 처리





Kubernetes 클러스터 아키텍처	물리적 구조: Master Node (Control Plane), Worker Node 1	시스템의고가용성 확보 구조를 한눈에 보여줌.
	논리적 요소: Ingress Controller, Service, Pods	트래픽이 들어와 컨테이너에 도달하는 흐름(Flow)을 명확히 제시하여 기술적 이해도를 높임.
	외부 접속: Ingress를 통한 외부(User) 연결 명시	네트워크 구조가 정확히 작동함을 시각적으로 증명함.

3.2 주요 구성 요소 상세

3.2.1 Frontend (Nginx + React)

구분	내용
기술 스택	React 18, Vite, Nginx
역할	사용자 인터페이스(UI/UX) 제공 및 백엔드 API 통신
배포 전략	빌드된 정적 파일을 Nginx 서빙
기술적 효과	레이턴시 최적화 및 백엔드 서비스 부하 경감
리소스 할당	CPU 50m, Memory 64Mi (초경량 구성)

3.2.2 Backend (FastAPI)

구분	내용
기술 스택	Python 3.11, FastAPI, SQLAlchemy
역할	사용자 인증, 파일 파싱, Claude API 연동, 데이터베이스 관리 (핵심 비즈니스 로직)
선택 근거	FastAPI 비동기 I/O (Async I/O) 활용
기술적 효과	적은 리소스 (CPU 250m)로 높은 동시 요청 처리 성능 제공 (Cloud-Native 최적화)
주요 API	/auth/signup, /auth/login, /{project_id}/summarize 등

3.2.3 Database (SQLite + PVC)

- 기술 선택: SQLite (파일 기반 DB)
- 영속성 보장: PVC (PersistentVolumeClaim) 활용 (저장 경로: 워커 노드 /mnt/data/sqlite)
- 선택 근거 (자원 효율화):
 - 일반 RDB 컨테이너 대비 추가 메모리 자원 소모 최소화 (메모리 0Mi 추가 소모)
- 기술적 한계 (Critical Note):
 - PoC 환경에 적합하나, 높은 동시성 환경에서 트랜잭션 충돌 발생 가능성
 - 실무 환경 전환 시 클라우드 관리형 DB (RDS) 분리 필수

3.2.4 Kubernetes 리소스 활용 현황

리소스	기술적 역할
Namespace (claude-app)	리소스 격리 및 관리 일관성 확보
Secret	API 키 등 민감 정보의 암호화 저장 및 Pod 주입
Service (ClusterIP)	Frontend-Backend 간 내부 통신 경로 제공
Ingress	외부 접근 시 L7 라우팅 게이트웨이 역할 수행
PVC/PV	DB 파일 저장을 통한 데이터 영속성 확보

3.3 기술적 특징 및 최적화 전략

3.3.1 Cloud-Native 자원 효율화 전략

- 목표: 제한된 환경에서 다수의 마이크로서비스 효율적 운영
- 결과: 기존 RDB 포함 구성 대비 총 메모리 사용량 50% 이하 절감

구성 요소	메모리 할당량	최적화 방법 참석
Frontend	64Mi	Nginx 정적 서빙
Backend	256Mi	FastAPI 비동기 처리
Database	1Gi (추가)	SQLite 파일 기반 데이터베이스

```

20-backend > ! deployment.yaml
 6 spec:
11   template:
15     spec:
16       containers:
17         - name: backend
33           volumeMounts:
34             - mountPath: /app/data
35               name: sqlite-storage
36           resources:
37             requests:
38               memory: "256Mi"
39               cpu: "250m"
40             limits:
41               memory: "512Mi"
42               cpu: "500m"
43           volumes:
44             - name: sqlite-storage
45               persistentVolumeClaim:
46                 claimName: sqlite-pvc

10-database > ! storage.yaml
 1 apiVersion: v1
 2 kind: PersistentVolume
 3 metadata:
 4   name: sqlite-pv
 5   labels:
 6     type: local
 7 spec:
 8   storageClassName: manual
 9   capacity:
10     storage: 1Gi
11   accessModes:
12     - ReadWriteOnce
13   hostPath:
14     path: "/mnt/data/sqlite" # 워크 노드의 이 경로에 파일이 실제로 저장됨
15 ---
16 apiVersion: v1
17 kind: PersistentVolumeClaim
18 metadata:
19   name: sqlite-pvc
20   namespace: claude-app
21 spec:
22   storageClassName: manual
23   accessModes:
24     - ReadWriteOnce
25   resources:
26     requests:
27       storage: 1Gi

30-frontend > ! deployment.yaml
 6 spec:
11   template:
15     spec:
16       containers:
17         - name: frontend
18           # *** 모든 컨테이너는 반드시 이 이미지 사용 ***
19           image: leeon3345/frontend:v9
20           imagePullPolicy: Always
21           ports:
22             - containerPort: 80
23           resources:
24             requests:
25               memory: "64Mi" # Nginx
26               cpu: "50m"
27             limits:
28               memory: "128Mi" # 128MB면 충분
29               cpu: "200m"

```

3.3.2 외부 LLM 서비스 의존성 관리

- 모델 및 방식: Anthropic Claude API (claude-sonnet-4-20250514)를 HTTP/S 비동기 요청으로 호출
- 기술적 의의:

- AI 연산을 위한 로컬 GPU 자원 완전 제거
- 인프라와 AI 모델 연산 환경의 완벽한 분리
- LLM 모델 변경에 대한 유연한 대응 체계 확보

3.3.3 문서 파일 파싱 기술

파일 형식	사용 라이브러리
PDF	PyPDF2
DOCX	python-docx
TXT	기본 파일 읽기 (file read)

IV. 비교 분석 및 비판적 시각

4.1 유사 OSS 비교 분석

4.1.1 컨테이너 오케스트레이션 도구 비교

항목	Kubernetes (선택)	Docker Swarm	Nomad
학습 곡선	높음 (복잡한 개념)	낮음 (쉬운 접근성)	중간 (단일 바이너리)
기능/확장성	매우 풍부 (Deployment, Service Mesh 등)	기본적 (단순 클러스터링)	유연함 (배치 스케줄링 특화)
커뮤니티	매우 활발 (압도적)	감소 추세	중간
자동 스케일링	HPA (Horizontal Pod Autoscaler) 지원	제한적 (매뉴얼 조정)	지원 (리소스 기반)
Self-Healing	강력 (컨트롤 루프 기반)	기본	중간
엔터프라이즈 채택률	96% (업계 표준)	감소	증가

4.1.2 Kubernetes 선정의 기술적 정당성

- 업계 표준 (De-facto Standard): 클라우드 네이티브 컴퓨팅 파운데이션(CNCF)의 핵심 프로젝트로, 엔터프라이즈 환경의 표준 인프라로 확립

- 풍부한 생태계: Helm (패키징), Prometheus/Grafana (모니터링), Istio (Service Mesh) 등 광범위한 확장 도구와 연동 용이
- 클라우드 중립성: AWS, GCP, Azure, On-Premise 등 모든 환경에서 일관된 배포 및 운영 환경 제공
- 선언적 구성 (Declarative Configuration): 원하는 최종 상태를 정의하여 시스템 운영의 일관성 및 안정성 확보

4.2 데이터베이스 선택 비교 및 한계 분석

4.2.1 데이터베이스 솔루션 비교

항목	SQLite (선택)	PostgreSQL	MySQL
메모리 사용	~0MB	~500MB 이상	~400MB 이상
동시 접속 처리	제한적	우수	우수
적합한 규모	소규모, PoC	대규모, 고가용성	중~대규모

4.2.2 SQLite 선택의 기술적 이유 및 한계

- 선택 이유: 제한된 8GB RAM 환경에서 자원 효율화를 극대화하고, 프로토타입 단계의 빠른 구축을 위함.
- 한계점: 동시성 문제 및 수평 확장(Scale-out) 불가. 프로덕션 전환 시 Kubernetes StatefulSet 기반 RDB 전환 필수.

4.3 비판적 분석 및 로드맵

4.3.1 프로젝트의 기술적 장점

- 리소스 효율성 극대화: 경량 컴포넌트 및 SQLite 채택.
- 빠른 개발 및 배포 사이클: Docker와 K8s Deployment를 통한 자동화된 배포 체계 구현.
- Kubernetes 핵심 기능 숙련: K8s의 주요 리소스(Deployment, Service, Ingress, PVC)를 MSA 환경에 성공적으로 적용 및 검증.

4.3.2 프로젝트의 기술적 단점

- SQLite 동시성 한계: 쓰기 작업 시 단일 프로세스 락(Lock)으로 가용성 저하.
- 단일 레플리카 구성: Frontend/Backend Pod이 1개로 구성되어 단일 장애점(SPOF) 존재.

4.3.3 미래 개선 방향: 프로덕션 레디니스 로드맵

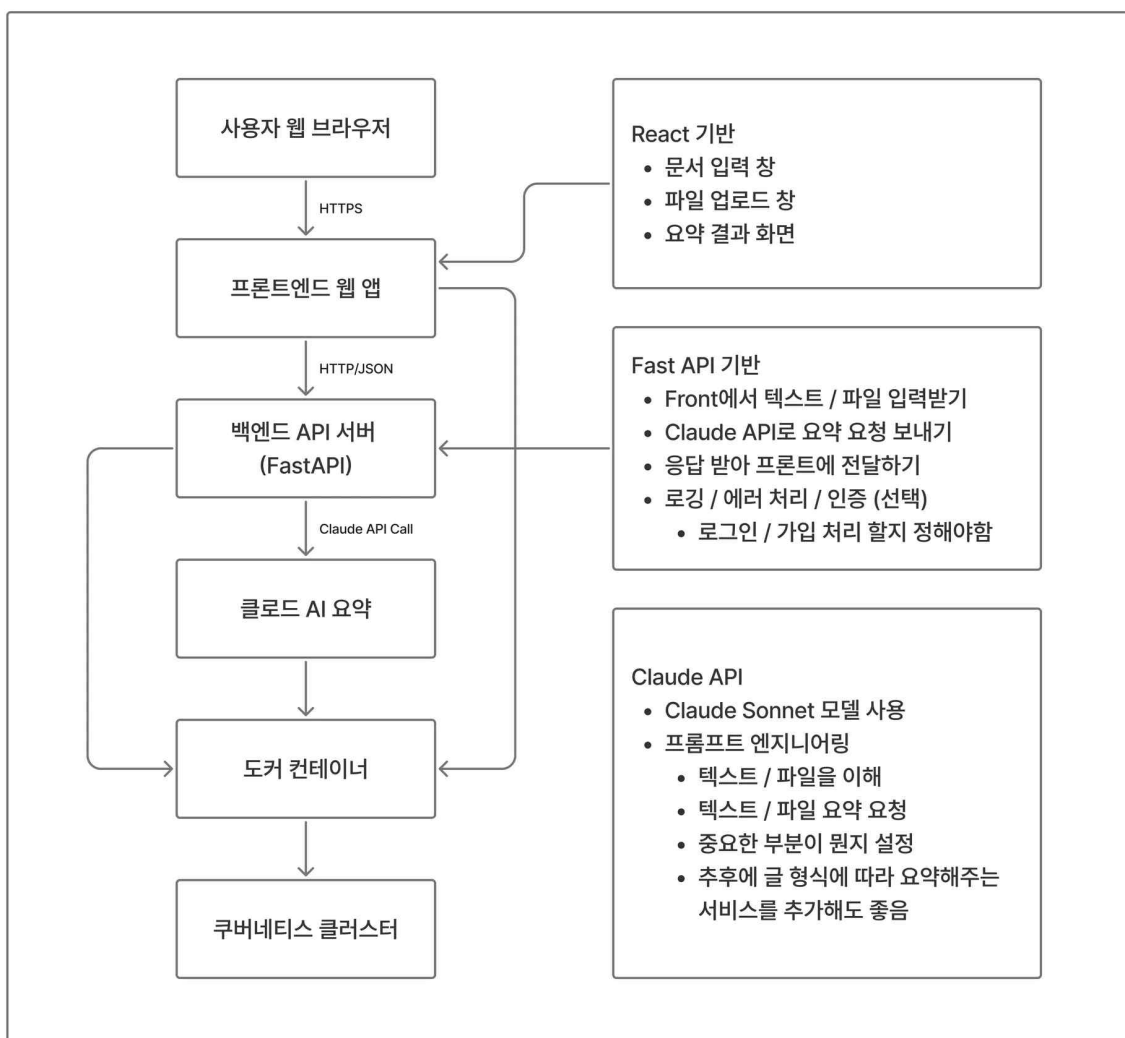
- 가용성 강화: Multi-replica 구성 (최소 3개 이상의 Pod)을 통한 단일 장애점 제거.

- 데이터베이스 안정화: PostgreSQL/MySQL StatefulSet 전환 및 K8s StatefulSet 리소스 활용.

V. 프로토타입 구현 및 검증

5.1 완성된 기능 및 Kubernetes 배포 현황

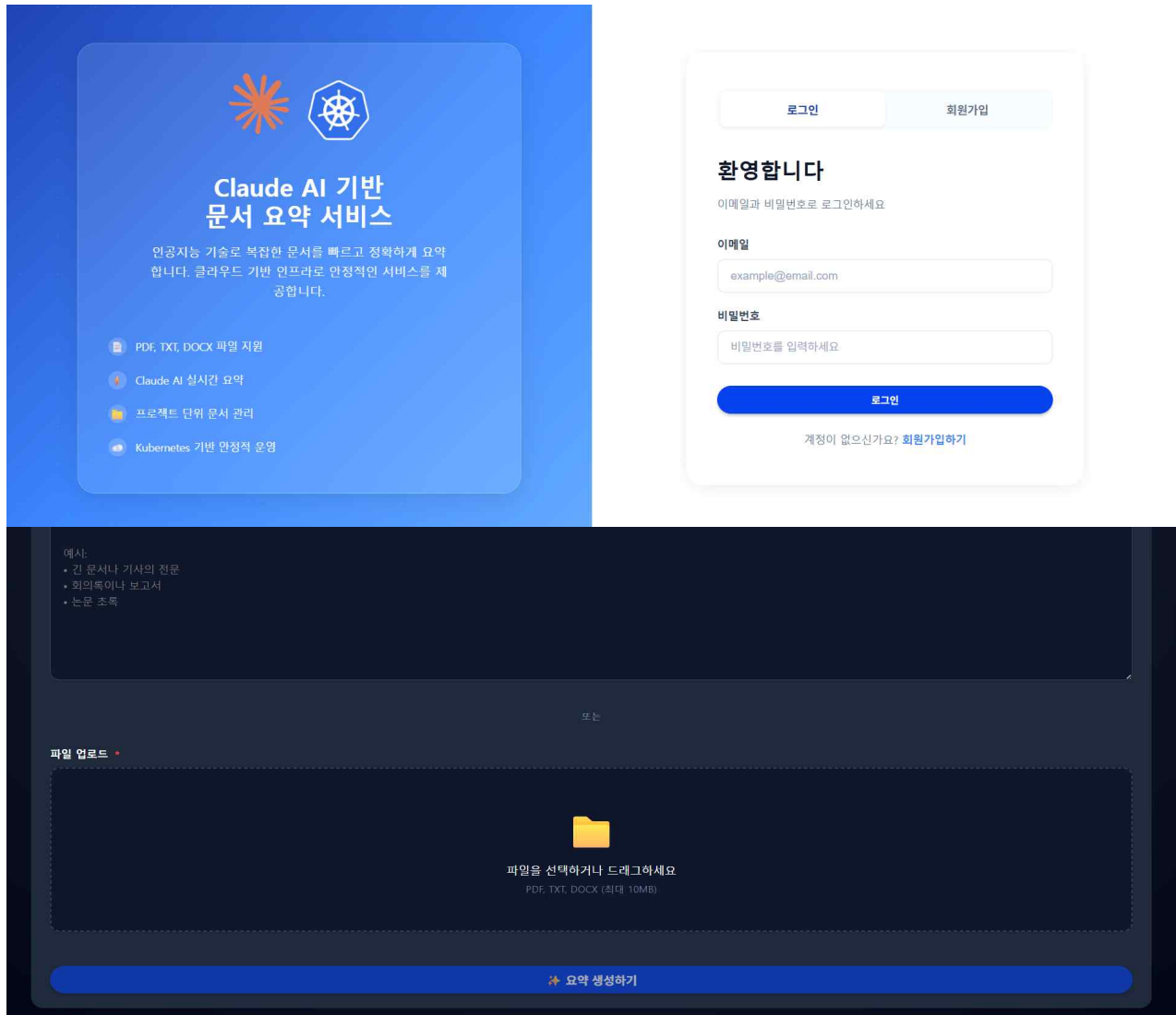
5.1.1 프로토타입 구조



5.1.2 핵심 기능 구현 결과

- 인증 및 관리: 회원가입 / 로그인 (JWT 인증), 프로젝트 생성 / 삭제.
- AI 서비스: 파일 업로드 (PDF, DOCX, TXT), Claude AI 요약 생성, 요약 히스토리 관리.

- 데이터 안정성: PersistentVolumeClaim (PVC)을 통한 데이터 영속성 확보.



5.1.3 Kubernetes 배포 완료 확인

항목	명령어	결과 요약
Pod 상태	\$ kubectl get pods -n claude-app	backend-api-xxx (1/1), frontend-app-xxx (1/1) Running 확인
Service 상태	\$ kubectl get svc -n claude-app	backend-service (ClusterIP), frontend-service (ClusterIP) 정상 작동

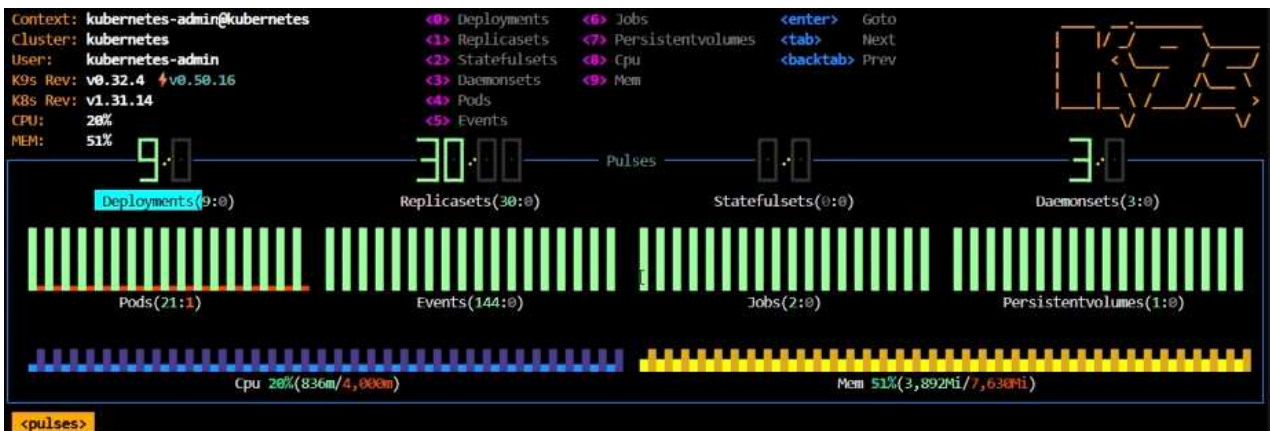
5.2 시스템 모니터링 및 리소스 사용 현황

5.2.1 클러스터 리소스 사용 분석

구분	측정 지표	사용 현황	기술적 분석
클러스터 CPU	4,000m 기준	20% (850m)	충분한 여유 자원 확보
클러스터 Memory	7,630Mi 기준	51% (3,923Mi)	운영 안정성 확보 (50% 이하 사용)
애플리케이션 CPU	총 90m	전체 클러스터의 약 2.2%	경량화 전략의 성공적인 증명
애플리케이션 Memory	총 225Mi	전체 클러스터 메모리의 약 3%	PostgreSQL 사용 시 대비 50% 이상 절감 달성

5.2.2 시스템 운영 현황

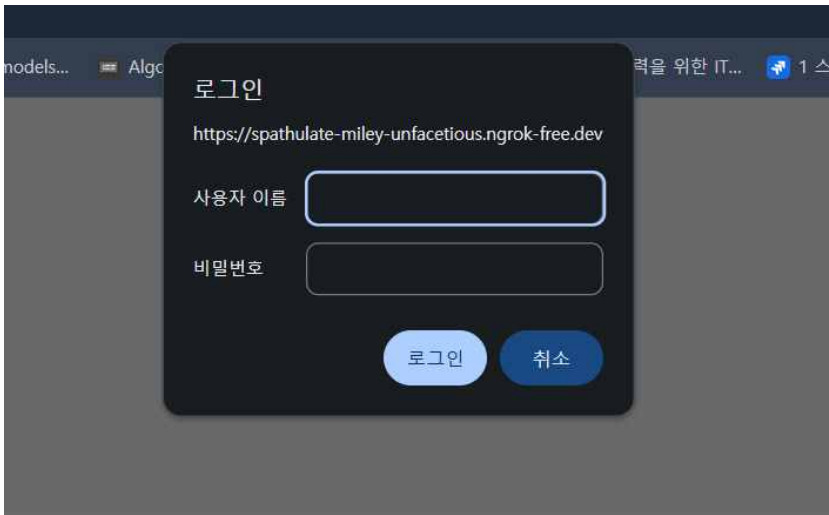
- 배포된 리소스: Deployments 9개, ReplicaSets 30개, Pods 21개, PersistentVolumes 1개.
- 분석: 전체 클러스터 자원의 극히 일부(CPU 2.2%, Memory 3%)만으로 애플리케이션을 안정적으로 운영하여, 리소스 최적화 전략의 성공적 구현을 검증.



5.3 보안 및 네트워크 구성 검증

5.3.1 Ingress Basic Auth 인증 (이중 보안 구성)

- 목표: 프로토타입 무분별한 외부 접근 차단 및 평가자 전용 접근 권한 부여.
- 필요성: 외부에 공개되는 프로토타입의 접근 통제(Access Control) 및 무단 사용 방지.
- 구현 방법:
 1. htpasswd 파일 생성 후 Kubernetes Secret (basic-auth)으로 등록.
 2. Ingress 리소스에 nginx.ingress.kubernetes.io/auth-type: basic Annotation 적용.
- 검증 결과:
 - 웹사이트 접속 시 로그인 팝업 자동 표시 확인.
 - 애플리케이션 로그인과 별개의 1차 보안 계층 확보.



5.3.2 데이터 영속성 보장 (PVC 전략 검증)

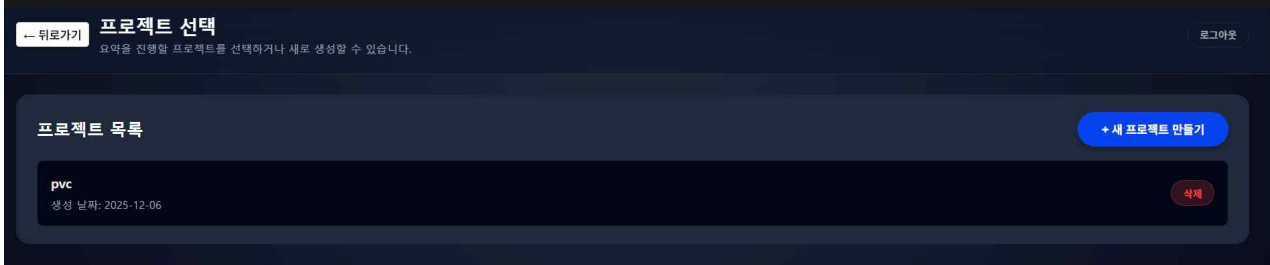
구분	내용
도전 과제	파일 기반 DB(SQLite) 사용으로 Pod 재시작 시 데이터 손실 문제 발생.
해결 방법	PersistentVolume (PV)과 PersistentVolumeClaim (PVC)을 사용하여 데이터를 워커 노드에 영구 저장하도록 구성.

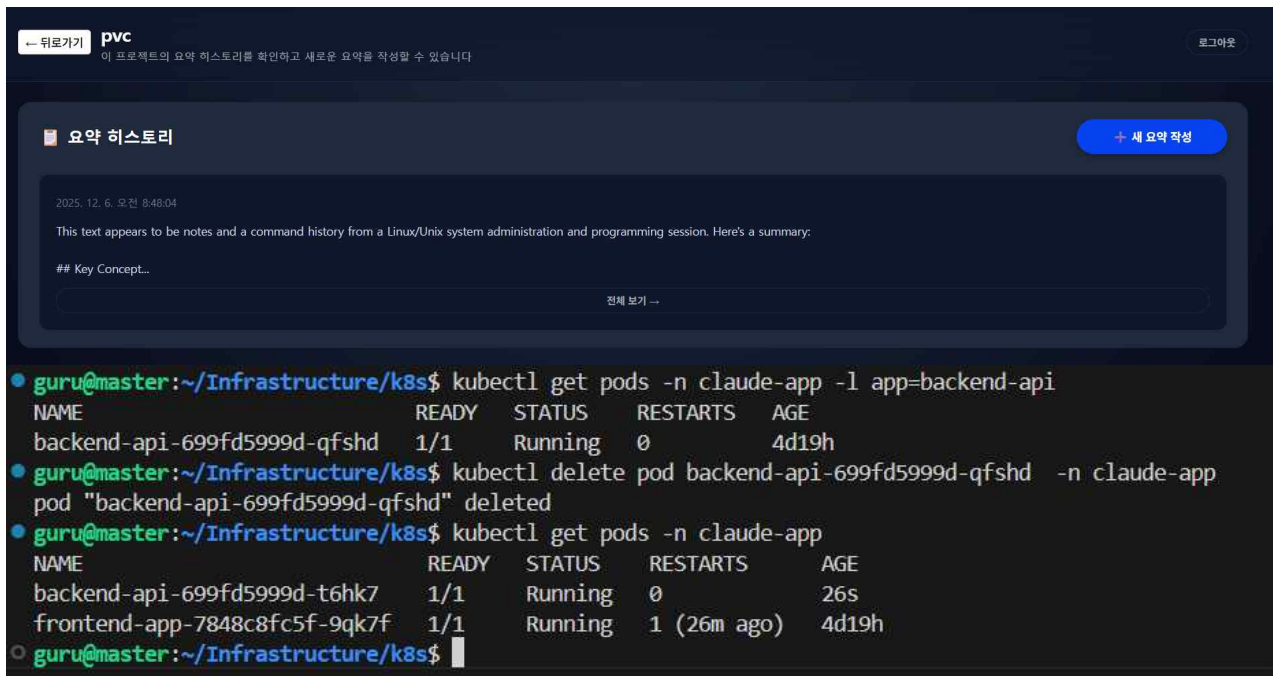
- PVC 테스트 시나리오:
 - 요약 데이터 생성 후 Backend Pod 강제 삭제 (kubectl delete pod).
 - Kubernetes의 새 Pod 자동 생성 후, 웹에서 요약 목록 확인.
- 최종 검증 결과:
 - 삭제 전 데이터가 재시작 후에도 그대로 보존되어, PVC 구성이 완벽하게 작동함 검증.

```

guru@master:~/Infrastructure/k8s$ kubectl get pvc -n claude-app
NAME          STATUS  VOLUME   CAPACITY   ACCESS MODES   STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
sqlite-pvc    Bound   sqlite-pv  1Gi        RWO            manual         <unset>                 15d
guru@master:~/Infrastructure/k8s$ kubectl get pods -n claude-app
NAME                                READY   STATUS    RESTARTS   AGE
backend-api-699fd5999d-qfshd        1/1     Running   0           4d19h
frontend-app-7848c8fc5f-9qk7f      1/1     Running   1 (17m ago) 4d19h
guru@master:~/Infrastructure/k8s$ kubectl get pods -n claude-app -l app=backend-api -o yaml | grep "mountPath: /app/data"
- mountPath: /app/data
- mountPath: /app/data
guru@master:~/Infrastructure/k8s$ kubectl exec -it -n claude-app $(kubectl get pod -n claude-app -l app=backend-api -o jsonpath='{.items[0].metadata.name}') -- ls -l /app/data
total 36
-rw-r--r-- 1 root root 32768 Dec 11 11:29 oss.db
guru@master:~/Infrastructure/k8s$

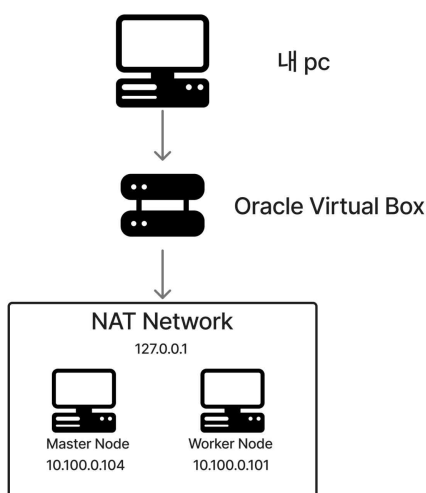
```





5.3.3 외부 접속 구성 (NAT 환경 극복)

- 도전 과제: IP가 고정된 NAT 네트워크 환경 특성상 외부 인터넷에서 내부 VM으로의 직접 접속 불가.
- 해결 방법: VirtualBox 포트 포워딩의 불안정성 문제를 극복하기 위해 ngrok와 kubectl 터널링을 결합한 하이브리드 터널링 방식 채택.
- 목표: 노트북 이동 환경(학교↔집)에서의 시연 및 평가 제약 해소 및 외부 접근성 확보.



호스트 전용 네트워크
NAT 네트워크
클라우드 네트워크

이름

IPv4 접두사

IPv6 접두사

DHCP 서버

k8s Network
10.100.0.0/24
fd17:625c:f037:6400...
사용함

일반 옵션(G)

포트 포워딩(P)

IPv4(4)

IPv6(6)

이름	프로토콜	호스트 IP	호스트 포트	게스트 IP	게스트 포트
ingre...	TCP	0.0.0.0	9090	10.100.0.101	80
master	TCP	127.0.0.1	104	10.100.0.104	22
node1	TCP	127.0.0.1	101	10.100.0.101	22

5.3.3.1 하이브리드 터널링 구현 단계

- Kubernetes 터널링 (Host PC ↔ VM):
 - Host PC 9090 포트를 Ingress Controller Service (80)로 직접 연결함.
 - 명령어: `$ kubectl port-forward -n ingress-nginx service/ingress-nginx-controller 9090:80 --address 0.0.0.0` 실행.
- 외부 공개 (Internet ↔ Host PC):
 - ngrok를 활용하여 로컬 9090 포트를 공인 도메인으로 노출함.
 - 명령어: `$ ngrok http 9090` 실행.
 - 외부 접근 주소: <https://team-project.ngrok-free.dev> 등으로 접근 가능하게 됨.

5.3.3.2 최종 네트워크 플로우

- 외부 요청이 Multi-Layer Tunneling을 통해 Ingress Basic Auth 및 서비스 라우팅으로 이어짐이 성공적으로 구현되었음 증명.
- 복잡한 네트워크 경로를 통해 Ingress Controller와 내부 서비스까지 안정적인 접근 경로 확보함.

5.4 보안 및 네트워킹 요약표

계층	구성 요소	목적	구현 방법
외부 접근	ngrok	외부 공개 및 임시 도메인 확보함	SSH Tunnel + 임시 도메인 활용됨
인증	Basic Auth	외부 접속 접근 제어 기능 수행됨	Ingress Annotation 적용됨
라우팅	Ingress	외부 트래픽 분산 처리됨	nginx-ingress-controller 활용됨
내부 통신	Service	Pod 간 안정적 통신 경로 제공됨	ClusterIP 방식 적용됨
데이터 보존	PVC/PV	영속성 보장 및 데이터 유지됨	hostPath (워커 노드) 활용됨

5.5 기술적 의사결정 요약

1. Basic Auth 추가:
 - 문제 인식: 프로토타입 공개 시 무분별한 접속 발생 우려됨.
 - 해결 방안: Ingress 레벨에서 접근 인증 추가함.
 - 효과: 평가자만 접속 가능하며, DDoS 공격 방어에 기여함.
2. PVC 전략 채택:
 - 문제 인식: SQLite 파일 기반으로 컨테이너 재시작 시 데이터 손실 발생됨.

- 해결 방안: PersistentVolume으로 워커 노드에 영구 저장함.
- 효과: Pod 삭제 및 재시작 시에도 데이터 보존 검증 완료됨.

3. ngrok 사용:

- 문제 인식: NAT 환경으로 공인 IP 확보 및 도메인 구매 불가함.
- 해결 방안: ngrok을 활용하여 임시 공개 URL 생성함.
- 효과: 외부에서 HTTPS 접속 가능성 확보됨 (프로토타입 용도).

5.6 데모 검증 및 결론

5.6.1 데모 시연 계획

1. Basic Auth 보안 데모:

- ngrok URL 접속 후 로그인 팝업 표시 확인됨.
- 잘못된 비밀번호 입력 시 401 에러 발생 확인됨.
- 올바른 계정 입력 시 서비스 접속 성공 확인됨.

2. PVC 영속성 데모:

- 요약 1개 생성 후 DB 저장 확인됨.
- k9s에서 backend Pod 강제 삭제 실행됨.
- 새 Pod 자동 생성 후 웹에서 요약 목록 데이터 보존 확인됨.

3. 리소스 효율성 데모:

- k9s pulse 모드 실행하여 리소스 사용 현황 공개됨.
- 전체 메모리의 3% 사용 강조 및 PostgreSQL 대비 효율성 설명됨.

4. 실제 기능 시연:

- 회원가입 및 로그인 절차 시연됨.
- 프로젝트 생성 및 PDF 파일 업로드 시연됨.
- Claude AI 요약 생성 (약 15초 소요) 및 요약 결과 확인 시연됨.

5.6.2 성능 측정 및 API 응답 시간

- 로그인: 약 200ms 소요됨.
- 목록 조회: 약 50ms 소요됨.
- 요약 생성: Claude API 응답 시간 포함 약 15초 소요됨.

VI. 팀워크 및 기여도

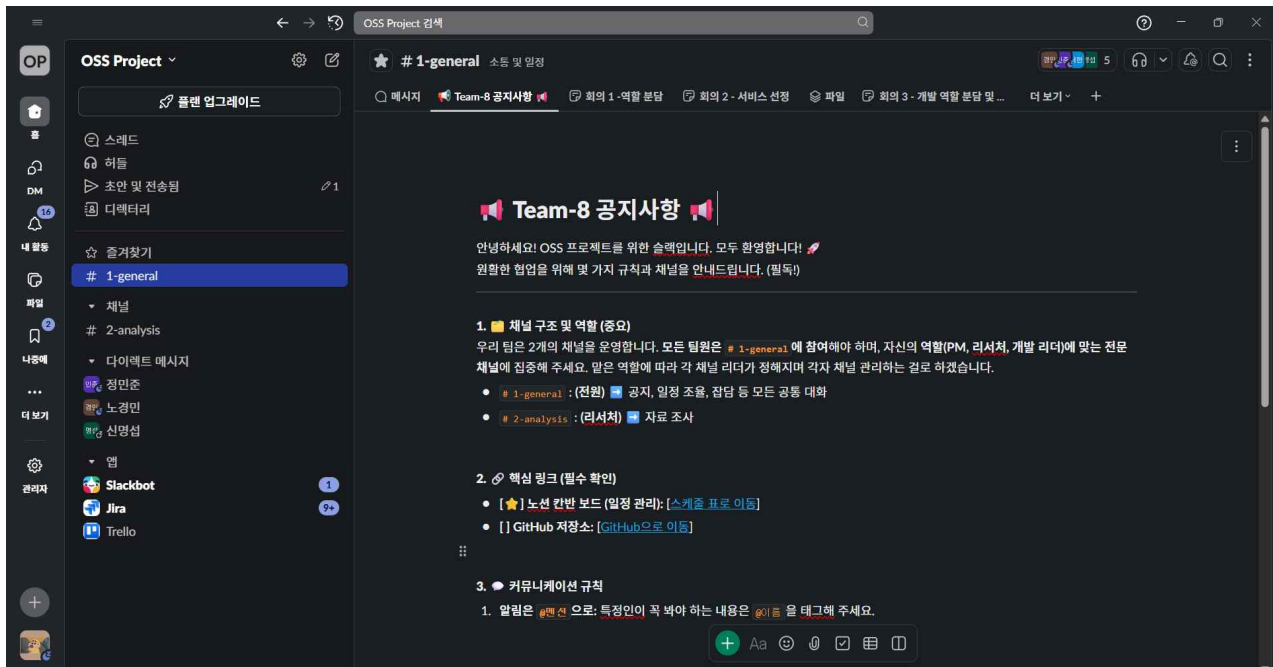
6.1 역할 분담

- 1단계: 대분류 역할 고정
- 2단계: 세부 과업 유동적 배분의 단계별 분업 시스템 적용
- 공동 작업: oss 후보 & 협업 툴 조사, 서비스 구상, 선정한 oss 조사, 웹 아키텍처 조사

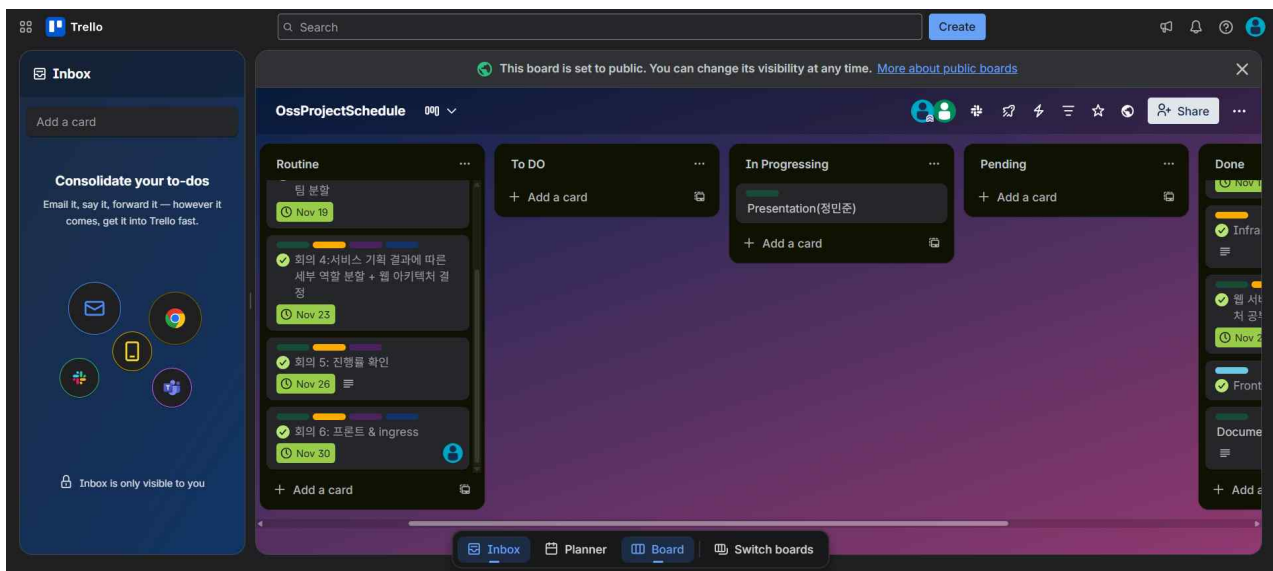
주 역할	담당자	주요 책임	세부 작업 분담
PM/쿠버네티스	이온	일정 및 회의 관리, 개발 총괄	[초기] - 협업 통과 OSS 선정을 위한 설문 조사 - 슬랙, 노션, 트렐로 환경 세팅 [중기] - 가상 머신 생성 및 클러스터 조인 - 쿠버네티스 인프라 조성 [후기] - 코드 리팩토링 및 테스트 - 시연 영상 및 시스템 아키텍처 다이어그램 제작
FE 개발/보고서	신명섭	기본 레이아웃 및 와이어프레임 구현	[초기] - 도입 배경 및 필요성 문서 작성 [중기] - 웹 애플리케이션 기본 레이아웃 및 와이어프레임 구현 [후기] - 기존 문서 통합 후 최종 보고서 작성
FE 개발	이서현	UI/UX	[초기] - 활용 사례 및 비교 문서 작성 [중기] - Figma 활용 UI 컴포넌트 설계 및 배치 [후기] - 페이지 UI/UX 디자인 적용
FE 개발/발표	정민준	기능 구현 및 API 연동 /발표	[초기] - 기술 아키텍처 문서 작성 [중기] - 백엔드와 API 연동 및 데이터 바인딩 수행 - Ingress Controller의 yaml 생성 [후기] - ppt 제작 및 발표
설계/BE 개발	노경민	서버 구축 및 데이터베이스 설계	[초기] - 깃헙 organization 팀 레포 관리 - 서비스 설계 및 개발 역할 분담 [중기] - 클라우드 API 리소스 관리 및 환경 세팅 - 서버 및 데이터베이스 생성 [후기] - 최종 검토

6.2 협업 툴

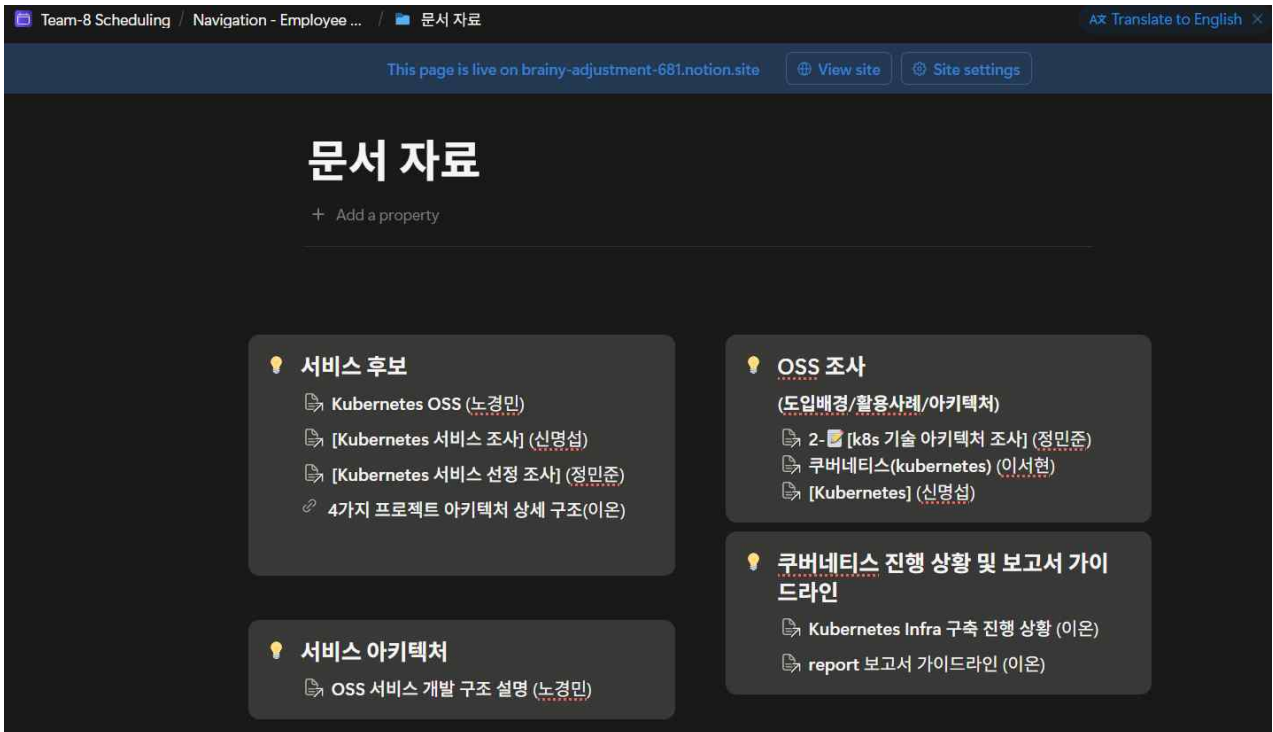
- 대화 및 회의: 슬랙



- 일정 관리: 트렐로



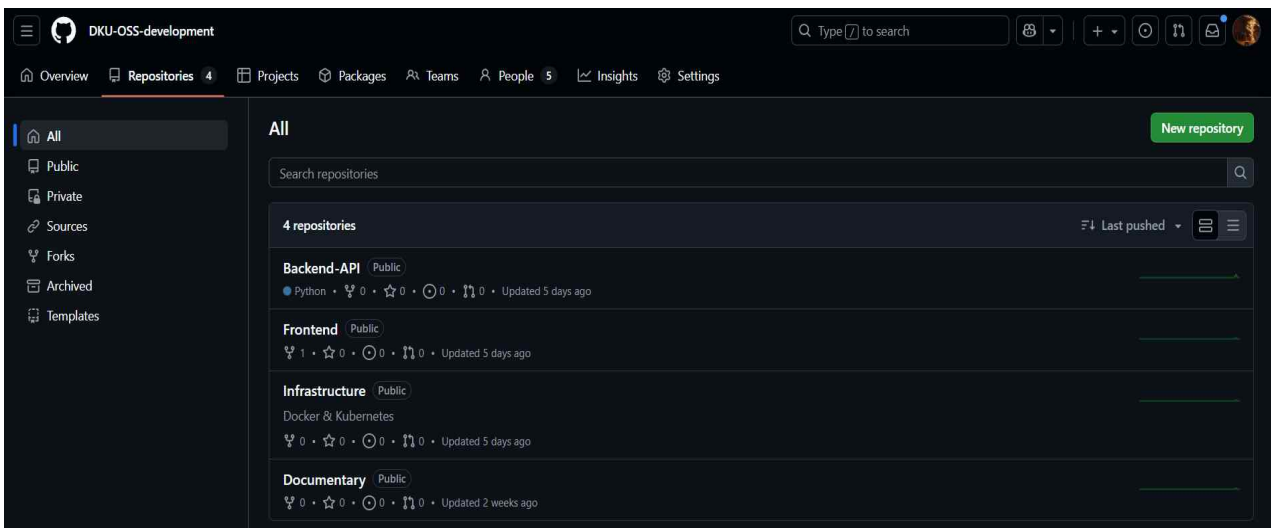
- 문서 관리: 노션



- 코드 관리: 깃헙

[GitHub Organization](https://github.com/orgs/DKU-OSS-development/repositories) : <https://github.com/orgs/DKU-OSS-development/repositories>

본 프로젝트는 다음 4개의 저장소로 구성됨:



[참고 자료]

공식 문서

1. Kubernetes Documentation: <https://kubernetes.io/docs/>
2. Docker Documentation: <https://docs.docker.com/>
3. Claude API Documentation: <https://docs.anthropic.com/>
4. FastAPI Documentation: <https://fastapi.tiangolo.com/>
5. React Documentation: <https://react.dev/>

기술 블로그

1. Netflix Tech Blog - Kubernetes 도입 사례
- <https://netflixtechblog.com/>
2. Spotify Engineering - Container Orchestration
- <https://engineering.atspotify.com/>
3. CNCF Blog - Cloud Native Trends 2024
- <https://www.cncf.io/blog/>
4. Kubernetes Blog
- <https://kubernetes.io/blog/>

컨퍼런스 발표

1. KubeCon 2024 - Best Practices for Resource Optimization
2. AWS re:Invent - Microservices Architecture
3. Google Cloud Next - Kubernetes at Scale

학술 자료

1. “Kubernetes: Up and Running” - Kelsey Hightower
2. “Designing Data-Intensive Applications” - Martin Kleppmann
3. CNCF Survey 2024 - Kubernetes Adoption Statistics